

P1: Digital Lockbox Lab Report

Andres Benitez & Luis Woo

September 24, 2026

Description

For project 1, we used the LCD display from A3 and the keypad from A2 and integrated them together. Once they were integrated, they were set up to create a virtual electronic lock. This lock starts in a locked state. In order to “unlock” the lock the correct 4 digit pin must be entered. Once the correct pin has been entered the user has the choice to “lock” the virtual lock by clicking any button on the keypad or change the pin to a new set of 4 numbers. When the lockbox is locked, the LED on the Nucleo board will be on and the LCD will display the “LOCKED ENTER KEY:” screen. When the lockbox is successfully unlocked, the LCD will display “UNLOCK PRESS KEY TO LOCK.”

Schematic Diagram

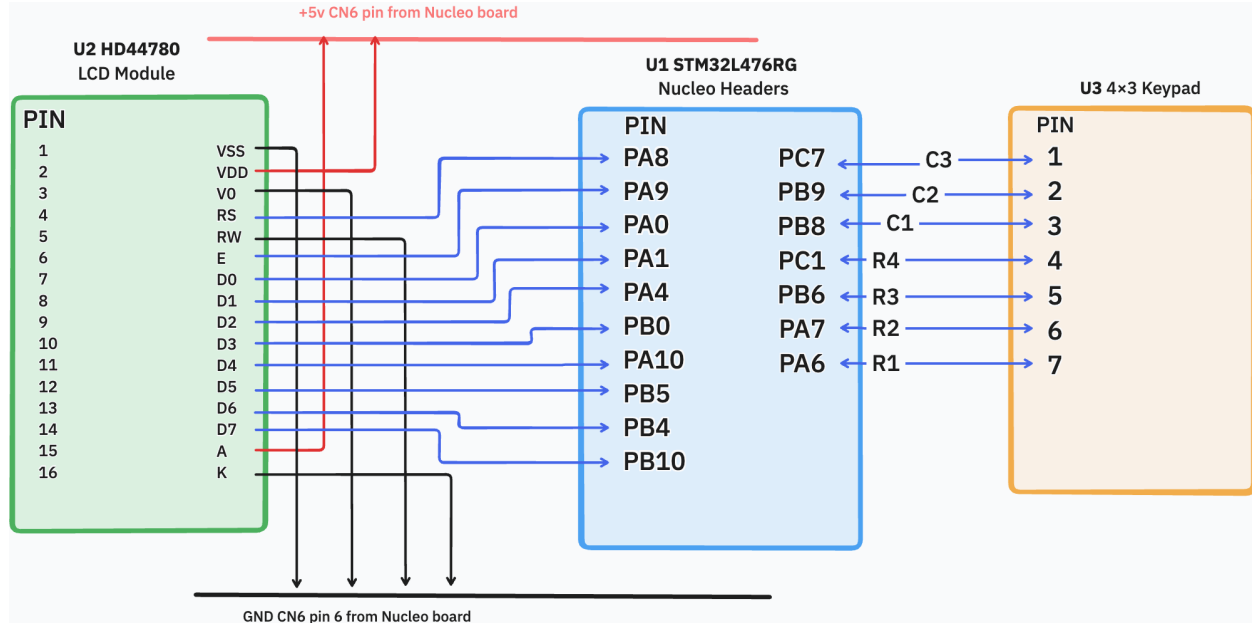


Figure 1: Schematic Diagrams. (Made in TLDRAW.com)

Software Structure

The structure of our software is split into three parts. The LCD driver (`lcd.c/lcd.h`) and handles GPIO, the 8-bit init sequence, and writing commands or characters to either row. The keypad driver (`keypad.c/keypad.h`) scans the rows and columns and returns 0–9, STAR, or POUND. `keypad.getkey()` waits for a press, debounces, and waits for the key to be released. The locking mechanism is only in `main.c`.

In `main.c`, we designed our system as a state machine with four states: LOCKED, UNLOCKED, SET_PIN, and CONFIRM. After the basic initialization, the state is locked with the LED on board turned on (Solid Green). Our main loop calls `keypad.getkey()` and a switch statement decides what the key does. We also implemented some helper functions to minimize the size of the main loop and to make our lives a little easier. `show_screen(line1, line2)` writes to both LCD rows, `add_digit(key)` stores the key pressed as ASCII, `clear_input()` resets the current entry, and `set_led(locked?)` turns the on-board LED on or off depending on the system's current state.

Source Code

```
1  #ifndef LCD_H
2  #define LCD_H
3
4  #include "main.h"
5
6  #define LCD_RS_PORT      GPIOA
7  #define LCD_RS_PIN      GPIO_PIN_8
8  #define LCD_E_PORT      GPIOA
9  #define LCD_E_PIN       GPIO_PIN_9
10
11
12 void lcd_init(void);
13 void lcd_command(uint8_t cmd);
14 void lcd_data(uint8_t ch);
15 void lcd_print(const char *s);
16 void lcd_set_line(uint8_t line);
17
18 #endif
```

Listing 1: (`lcd.h`)

```
1  #include "lcd.h"
2
3  static void lcd_gpio_init(void);
4  static void lcd_set_data(uint8_t data);
5  static void lcd_pulse_e(void);
6  static void lcd_write(uint8_t rs, uint8_t data);
7
8  /**
9   * @brief LCD GPIO as push-pull outputs. E / RS / data idle low.
10  */
11  static void lcd_gpio_init(void) {
12      RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOBEN;
```

```

13
14 // PA0, PA1, PA4, PA8 (RS), PA9 (E), PA10
15 GPIOA->MODER &= ~( (3U << 0) | (3U << 2) | (3U << 8) | (3U << 16)
16 | (3U << 18) | (3U << 20));
17
18 GPIOA->MODER |= ( (1U << 0) | (1U << 2) | (1U << 8) | (1U << 16)
19 | (1U << 18) | (1U << 20));
20
21 GPIOA->OTYPER &= ~(GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_4 |
22 GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10);
23
24 // PB0, PB4, PB5, PB10
25 GPIOB->MODER &= ~( (3U << 0) | (3U << 8) | (3U << 10) | (3U << 20));
26 GPIOB->MODER |= ( (1U << 0) | (1U << 8) | (1U << 10) | (1U << 20));
27 GPIOB->OTYPER &= ~(GPIO_PIN_0 | GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_10)
28 ;
29
30 GPIOA->BRR = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_4 |
31 GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10;
32 GPIOB->BRR = GPIO_PIN_0 | GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_10;
33 }
34
35 /**
36  * @brief Drive LCD D0-D7 from one byte.
37  */
38 static void lcd_set_data(uint8_t data) {
39     uint32_t a_set = 0;
40     uint32_t a_rst = 0
41     ;
42     uint32_t b_set = 0;
43     uint32_t b_rst = 0;
44
45     if (data & 0x01) {
46         a_set |= GPIO_PIN_0;
47     } else {
48         a_rst |= GPIO_PIN_0;
49     } // D0
50     if (data & 0x02) {
51         a_set |= GPIO_PIN_1;
52     } else {
53         a_rst |= GPIO_PIN_1;
54     } // D1
55     if (data & 0x04) {
56         a_set |= GPIO_PIN_4;
57     } else {
58         a_rst |= GPIO_PIN_4;
59     } // D2
60     if (data & 0x08) {
61         b_set |= GPIO_PIN_0;
62     } else {
63         b_rst |= GPIO_PIN_0;
64     } // D3
65     if (data & 0x10) {
66         a_set |= GPIO_PIN_10;

```

```

66     } else {
67         a_rst |= GPIO_PIN_10;
68     } // D4
69     if (data & 0x20) {
70         b_set |= GPIO_PIN_5;
71     } else {
72         b_rst |= GPIO_PIN_5;
73     } // D5
74     if (data & 0x40) {
75         b_set |= GPIO_PIN_4;
76     } else {
77         b_rst |= GPIO_PIN_4;
78     } // D6
79     if (data & 0x80) {
80         b_set |= GPIO_PIN_10;
81     } else {
82         b_rst |= GPIO_PIN_10;
83     } // D7
84
85     GPIOA->BSRR = a_set | (a_rst << 16);
86     GPIOB->BSRR = b_set | (b_rst << 16);
87 }
88
89 /**
90  * @brief Latch on the falling edge of E.
91  */
92 static void lcd_pulse_e(void) {
93     LCD_E_PORT->BSRR = LCD_E_PIN;    // E high
94     HAL_Delay(1);
95     LCD_E_PORT->BRR = LCD_E_PIN;    // E low, LCD captures here
96     HAL_Delay(1);
97 }
98
99 /**
100  * @brief Write a command (rs = 0) or character (rs = 1).
101  */
102 static void lcd_write(uint8_t rs, uint8_t data) {
103     if (rs) {
104         LCD_RS_PORT->BSRR = LCD_RS_PIN;
105     } else {
106         LCD_RS_PORT->BRR = LCD_RS_PIN;
107     }
108
109     lcd_set_data(data);
110     lcd_pulse_e();
111     HAL_Delay(2);
112 }
113
114 void lcd_command(uint8_t cmd) {
115     lcd_write(0, cmd);
116 }
117
118 void lcd_data(uint8_t ch) {
119     lcd_write(1, ch);

```

```

120 }
121
122 void lcd_print(const char *s) {
123     while (*s != '\0') {
124         lcd_data((uint8_t) *s);
125         s++;
126     }
127 }
128
129 // line 0 = first row (0x00), line 1 = second row (0x40)
130 void lcd_set_line(uint8_t line) {
131     lcd_command(line ? 0xC0 : 0x80);
132 }
133
134 /**
135  * @brief Initialize the LCD.
136  */
137 void lcd_init(void) {
138     lcd_gpio_init();
139
140     HAL_Delay(50);           // wait after power-up
141
142     lcd_command(0x30);       // 8-bit function set
143     HAL_Delay(5);
144     lcd_command(0x30);
145     HAL_Delay(1);
146     lcd_command(0x30);
147
148     lcd_command(0x38);       // 8-bit with 2 lines and 5x8 font
149     lcd_command(0x08);       // display off
150     lcd_command(0x01);       // clear
151     lcd_command(0x06);       // increment cursor
152     lcd_command(0x0F);       // display on / cursor on / blink on
153 }

```

Listing 2: (lcd.c)

```

1  #ifndef KEYPAD_H
2  #define KEYPAD_H
3
4  #include "main.h"
5
6  #define NUM_OF_ROWS 4
7  #define NUM_OF_COLS 3
8
9  #define STAR 10
10 #define POUND 11
11
12 void keypad_init(void);
13 int32_t keypad_read(void);
14 int32_t keypad_getkey(void);
15
16 #endif

```

Listing 3: (keypad.h)

```

1  #include "keypad.h"
2
3  typedef struct {
4      GPIO_TypeDef *port;
5      uint32_t pin;
6  } keypad_pin_t;
7
8  static const keypad_pin_t col_pin[NUM_OF_COLS] = {
9      { GPIOB, GPIO_PIN_8 },
10     { GPIOB, GPIO_PIN_9 },
11     { GPIOC, GPIO_PIN_7 },
12 };
13
14 static const keypad_pin_t row_pin[NUM_OF_ROWS] = {
15     { GPIOA, GPIO_PIN_6 },
16     { GPIOA, GPIO_PIN_7 },
17     { GPIOB, GPIO_PIN_6 },
18     { GPIOC, GPIO_PIN_1 },
19 };
20
21 static const int keymap[NUM_OF_ROWS][NUM_OF_COLS] = {
22     { 1, 2, 3 },
23     { 4, 5, 6 },
24     { 7, 8, 9 },
25     { STAR, 0, POUND },
26 };
27
28 static uint32_t pin_index(uint32_t pin) {
29     uint32_t i = 0;
30     while (((pin >> i) & 1U) == 0U) {
31         i++;
32     }
33     return i;
34 }
35
36 static void pin_output(GPIO_TypeDef *port, uint32_t pin) {
37     uint32_t i = pin_index(pin);
38     port->MODER &= ~(3U << (i * 2U));
39     port->MODER |= (1U << (i * 2U));
40     port->OTYPER &= ~pin;
41 }
42
43 static void pin_input_pulldown(GPIO_TypeDef *port, uint32_t pin) {
44     uint32_t i = pin_index(pin);
45     port->MODER &= ~(3U << (i * 2U));
46     port->PUPDR &= ~(3U << (i * 2U));
47     port->PUPDR |= (2U << (i * 2U));
48 }
49
50 static void cols_all_high(void) {
51     for (int32_t c = 0; c < NUM_OF_COLS; c++) {
52         col_pin[c].port->BSRR = col_pin[c].pin;
53     }

```

```

54 }
55
56 static int32_t row_is_high(int32_t r) {
57     return (row_pin[r].port->IDR & row_pin[r].pin) != 0U;
58 }
59
60 static int32_t any_row_high(void) {
61     for (int32_t r = 0; r < NUM_OF_ROWS; r++) {
62         if (row_is_high(r)) {
63             return 1;
64         }
65     }
66     return 0;
67 }
68
69 void keypad_init(void) {
70     RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOBEN
71         | RCC_AHB2ENR_GPIOCEN;
72
73     for (int32_t r = 0; r < NUM_OF_ROWS; r++) {
74         pin_input_pullup(row_pin[r].port, row_pin[r].pin);
75     }
76     for (int32_t c = 0; c < NUM_OF_COLS; c++) {
77         pin_output(col_pin[c].port, col_pin[c].pin);
78     }
79     // All columns high so any press pulls its row high
80     cols_all_high();
81 }
82
83 int32_t keypad_read(void) {
84     if (!any_row_high()) {
85         return -1;
86     }
87
88     int32_t key = -1;
89
90     for (int32_t c = 0; c < NUM_OF_COLS; c++) {
91         for (int32_t i = 0; i < NUM_OF_COLS; i++) {
92             col_pin[i].port->BRR = col_pin[i].pin;
93         }
94         col_pin[c].port->BSRR = col_pin[c].pin;
95         HAL_Delay(1);
96
97         for (int32_t r = 0; r < NUM_OF_ROWS; r++) {
98             if (row_is_high(r)) {
99                 key = keymap[r][c];
100             }
101         }
102     }
103
104     cols_all_high();
105     return key;
106 }
107

```

```

108 int32_t keypad_getkey(void) {
109     int32_t key;
110
111     do {
112         key = keypad_read();
113     } while (key < 0);
114
115     HAL_Delay(20);
116     key = keypad_read();
117
118     while (any_row_high()) {
119     }
120
121     HAL_Delay(20);
122     return key;
123 }

```

Listing 4: (keypad.c)

```

1  #include "main.h"
2  #include "lcd.h"
3  #include "keypad.h"
4
5  void SystemClock_Config(void);
6  static void MX_GPIO_Init(void);
7
8  void show_screen(const char *line1, const char *line2);
9  void clear_input(void);
10 void show_locked(void);
11 void show_unlocked(void);
12 void set_led(int locked);
13 void show_set_pin(void);
14 void show_confirm(void);
15 void show_saved(void);
16 void add_digit(int32_t key);
17 void unlock_animation(void);
18
19 typedef enum {
20     LOCKED, UNLOCKED, SET_PIN, CONFIRM,
21 } state_t;
22
23 static uint8_t lock_closed[8] = {
24     0b01110, //   ***
25     0b10001, //  *   *
26     0b10001, //  *   *
27     0b11111, //  *****
28     0b11011, //  **  **
29     0b11011, //  **  **
30     0b11111, //  *****
31     0b00000 //
32 };
33
34 static state_t state = LOCKED;
35 static uint32_t curr_pin = DEFAULT_PIN;

```

```

36 static uint32_t new_pin = 0;
37 static uint32_t input = 0;
38 static uint8_t digits = 0;
39 static char typed[5];
40
41 int main(void) {
42
43     HAL_Init();
44     SystemClock_Config();
45     MX_GPIO_Init();
46
47     lcd_init();
48     keypad_init();
49
50     BSP_LED_Init(LED_GREEN);
51     /* Initialize USER push-button, will be used to trigger an interrupt
52     each time it's pressed.*/
53     BSP_PB_Init(BUTTON_USER, BUTTON_MODE_EXTI);
54
55     typed[0] = '\0';
56
57     BSP_LED_On(LED_GREEN);
58     show_locked();
59     set_led(LOCKED);
60
61     while (1) {
62         int32_t key = keypad_getkey();
63
64         switch (state) {
65
66             case LOCKED: // Checks 4 pin input
67                 if (key == STAR) {
68                     clear_input();
69                     show_locked();
70                     set_led(LOCKED);
71                     state = LOCKED;
72                 } else if (key >= 0 && key <= 9 && digits < PIN_LEN) {
73                     add_digit(key);
74                     show_locked();
75                     if (digits == PIN_LEN) {
76                         if (input == curr_pin) {
77                             state = UNLOCKED;
78                             clear_input();
79                             set_led(UNLOCKED);
80                             unlock_animation();
81                             show_unlocked();
82                         } else {
83                             clear_input();
84                             show_locked();
85                         }
86                     }
87                 }
88                 break;

```

```

89     case UNLOCKED: // Any key press re-locks, '#' for new PIN
90         if (key == POUND) {
91             clear_input();
92             state = SET_PIN;
93             show_set_pin();
94         } else {
95             clear_input();
96             state = LOCKED;
97             set_led(state);
98             show_locked();
99         }
100         break;
101
102     case SET_PIN: // Sets a new 4-digit PIN
103         if (key == STAR) {
104             clear_input();
105             show_set_pin();
106         } else if (key >= 0 && key <= 9 && digits < PIN_LEN) {
107             add_digit(key);
108             show_set_pin();
109             if (digits == PIN_LEN) {
110                 new_pin = input;
111                 clear_input();
112                 state = CONFIRM;
113                 show_confirm();
114             }
115         }
116         break;
117
118     case CONFIRM: // Confirms new 4-digit PIN
119
120         if (key == STAR) {
121             clear_input();
122             show_confirm();
123         } else if (key >= 0 && key <= 9 && digits < PIN_LEN) {
124             add_digit(key);
125             show_confirm();
126             if (digits == PIN_LEN) {
127                 if (input == new_pin) {
128                     curr_pin = new_pin;
129                     clear_input();
130                     state = LOCKED;
131                     set_led(LOCKED);
132                     show_saved();
133                     HAL_Delay(1000);
134                     show_locked();
135                 } else {
136                     clear_input();
137                     state = SET_PIN;
138                     show_set_pin();
139                 }
140             }
141         }
142         break;

```

```

43         default:
44             break;
45     }
46
47 }
48 /* USER CODE END 3 */
49 }
50
51 void show_screen(const char *line1, const char *line2) {
52     lcd_command(0x01); // Clear
53     HAL_Delay(2);
54
55     lcd_set_line(0);
56     lcd_print(line1);
57     lcd_set_line(1);
58     lcd_print(line2);
59
60 }
61
62 void clear_input(void) {
63     input = 0;
64     digits = 0;
65     typed[0] = '\0';
66 }
67
68 void show_locked(void) {
69     show_screen("LOCKED", "ENTER KEY:");
70     lcd_print(typed);
71     lcd_command(0x0F); // cursor / blink back on
72
73 }
74
75 void show_unlocked(void) {
76     show_screen("UNLOCKED!!!", "PRESS KEY TO");
77     uint8_t slot = 1;
78     uint8_t cgram_address = slot * 8; // slot starts at byte 8
79
80
81     lcd_command(0x40 | cgram_address); // set cgram_address
82
83     for (int i = 0; i < 8; i++) {
84         lcd_data(lock_closed[i]);
85     }
86
87     lcd_command(0x80 | (0x40 + 13)); // row 2, column 13
88     lcd_data(slot);
89     lcd_command(0x0C); // set blinker off
90 }
91
92 void show_set_pin(void) {
93     show_screen("NEW PIN", "");
94     lcd_print(typed);
95 }
96

```

```

197 void show_confirm(void) {
198     show_screen("CONFIRM", "");
199     lcd_print(typed);
200 }
201
202 void show_saved(void) {
203     show_screen("NEW PIN SAVED", "");
204 }
205
206 void set_led(int locked) {
207     if (locked == LOCKED) {
208         BSP_LED_On(LED_GREEN);
209     } else {
210         BSP_LED_Off(LED_GREEN);
211     }
212 }
213
214 void add_digit(int32_t key) {
215     typed[digits] = (char) (key + 0x30);
216     digits++;
217     typed[digits] = '\0';
218     input = input * 10 + key;
219 }
220
221 void unlock_animation(void) {
222     const char *frames[] = {
223         "[          ]",
224         "[**         ]",
225         "[*****       ]",
226         "[*****      ]",
227         "[*****     ]", };
228
229     lcd_command(0x0C); // display on, cursor off
230     for (int i = 0; i < 5; i++) {
231         show_screen("UNLOCKING", frames[i]);
232         HAL_Delay(100);
233     }
234     lcd_command(0x0F); // cursor / blink back on
235 }
236
237 /**
238  * @brief System Clock Configuration
239  * @retval None
240  */
241
242 void SystemClock_Config(void) {
243     RCC_OscInitTypeDef RCC_OscInitStruct = { 0 };
244     RCC_ClkInitTypeDef RCC_ClkInitStruct = { 0 };
245
246     /** Configure the main internal regulator output voltage
247      */
248     if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1)
249         != HAL_OK) {
250         Error_Handler();

```

```

251     }
252
253     /** Initializes the RCC Oscillators according to the specified
254     parameters
255     * in the RCC_OscInitTypeDef structure.
256     */
257     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
258     RCC_OscInitStruct.HSISState = RCC_HSI_ON;
259     RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
260     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
261     RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
262     RCC_OscInitStruct.PLL.PLLM = 1;
263     RCC_OscInitStruct.PLL.PLLN = 10;
264     RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV7;
265     RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
266     RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
267     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK) {
268         Error_Handler();
269     }
270
271     /** Initializes the CPU, AHB and APB buses clocks
272     */
273     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK |
274         RCC_CLOCKTYPE_SYCLK
275         | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
276     RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
277     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
278     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
279     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
280
281     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4) != HAL_OK
282         ) {
283         Error_Handler();
284     }
285 }
286
287 /**
288 * @brief GPIO Initialization Function
289 * @param None
290 * @retval None
291 */
292 static void MX_GPIO_Init(void) {
293     GPIO_InitTypeDef GPIO_InitStruct = { 0 };
294     /* USER CODE BEGIN MX_GPIO_Init_1 */
295
296     /* USER CODE END MX_GPIO_Init_1 */
297
298     /* GPIO Ports Clock Enable */
299     __HAL_RCC_GPIOC_CLK_ENABLE();
300     __HAL_RCC_GPIOH_CLK_ENABLE();
301     __HAL_RCC_GPIOA_CLK_ENABLE();
302     __HAL_RCC_GPIOB_CLK_ENABLE();
303
304     /*Configure GPIO pins : USART_TX_Pin USART_RX_Pin */

```

```

802     GPIO_InitStruct.Pin = USART_TX_Pin | USART_RX_Pin;
803     GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
804     GPIO_InitStruct.Pull = GPIO_NOPULL;
805     GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
806     GPIO_InitStruct.Alternate = GPIO_AF7_USART2;
807     HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
808
809     /* USER CODE BEGIN MX_GPIO_Init_2 */
810
811     /* USER CODE END MX_GPIO_Init_2 */
812 }
813
814 /* USER CODE BEGIN 4 */
815
816 /* USER CODE END 4 */
817
818 /**
819  * @brief This function is executed in case of error occurrence.
820  * @retval None
821  */
822 void Error_Handler(void) {
823     /* USER CODE BEGIN Error_Handler_Debug */
824     /* User can add his own implementation to report the HAL error return
825     state */
826     __disable_irq();
827     while (1) {
828
829     /* USER CODE END Error_Handler_Debug */
830 }
831
832 #ifdef USE_FULL_ASSERT
833 /**
834  * @brief Reports the name of the source file and the source line number
835  * where the assert_param error has occurred.
836  * @param file: pointer to the source file name
837  * @param line: assert_param error line source number
838  * @retval None
839  */
840 void assert_failed(uint8_t *file, uint32_t line)
841 {
842     /* USER CODE BEGIN 6 */
843     /* User can add his own implementation to report the file name and line
844     number,
845     ex: printf("Wrong parameters value: file %s on line %d\r\n", file,
846     line) */
847     /* USER CODE END 6 */
848 }
849 #endif /* USE_FULL_ASSERT */

```

Listing 5: (main.c)